

MODUL PRAKTIKUM

MATA KULIAH STRUKTUR DATA DAN ALGORITMA

PERTEMUAN 2

SEMESTER GENAP

TAHUN AJARAN 2025 - 2026



Disusun oleh:

Nisa Dwi Angresti, M.Kom.

Farhan Fitrahadi

Mikail Samyth Habibillah

Muhammad Habib

Putri Diva Riyanti

DEPARTEMEN SISTEM INFORMASI

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

IDENTITAS PRAKTIKUM

IDENTITAS MATA KULIAH

Kode mata kuliah	JSI62108
Nama mata kuliah	STRUKTUR DATA DAN ALGORITMA
CPMK yang dibebankan pada praktikum	CPMK-01 Mampu mengidentifikasi dan memformulasikan permasalahan organisasi yang berkaitan dengan penerapan prinsip, konsep, serta metode dalam struktur data dan algoritma sebagai dasar untuk penyusunan solusi
Materi Praktikum Pertemuan 2	Pengantar Linked List
	Singly Linked List
	Circular Linked List
	Doubly Linked List
	Circular Doubly Linked Lists

IDENTITAS DOSEN DAN ASISTEN MAHASISWA

Nama Dosen Pengampu	1. Prof. Ir. Surya Afnarius, MSc, PhD 2. Nisa Dwi Angresti, M.Kom.
Nama Asisten Mahasiswa (Kelas A)	1. 2311522037 - Farhan Fitrahadi 2. 2311523013 - Putri Diva Riyanti 3. 2411521001 - Dhyva Aulia Hendri 4. 2411521012 – Anggun Meika Candra 5. 2411522004 - Ferdian Rahman 6. 2411522019 - Martia Perdana Putri 7. 2411522024 - Muhammad Habib 8. 2411523016 - Mikail Samyth Habibillah
Nama Asisten Mahasiswa (Kelas B)	1. 2311522037 - Farhan Fitrahadi 2. 2311523013 - Putri Diva Riyanti 3. 2411521001 - Dhyva Aulia Hendri 4. 2411521012 – Anggun Meika Candra 5. 2411522004 - Ferdian Rahman 6. 2411522019 - Martia Perdana Putri 7. 2411522024 - Muhammad Habib 8. 2411523016 - Mikail Samyth Habibillah
Nama Asisten Mahasiswa	1. 2311522037 - Farhan Fitrahadi

(Kelas C)	2. 2311523013 - Putri Diva Riyanti 3. 2411521001 - Dhyva Aulia Hendri 4. 2411521012 – Anggun Meika Candra 5. 2411522004 - Ferdian Rahman 6. 2411522019 - Martia Perdana Putri 7. 2411522024 - Muhammad Habib 8. 2411523016 - Mikail Samyth Habibillah
Nama Asisten Mahasiswa (Kelas D)	1. 2311522037 - Farhan Fitrahadi 2. 2311523013 - Putri Diva Riyanti 3. 2411521001 - Dhyva Aulia Hendri 4. 2411521012 – Anggun Meika Candra 5. 2411522004 - Ferdian Rahman 6. 2411522019 - Martia Perdana Putri 7. 2411522024 - Muhammad Habib 8. 2411523016 - Mikail Samyth Habibillah

DAFTAR ISI

IDENTITAS PRAKTIKUM.....	2
IDENTITAS MATA KULIAH.....	2
IDENTITAS DOSEN DAN ASISTEN MAHASISWA.....	2
DAFTAR ISI.....	4
LINKED LIST.....	5
A. PERSIAPAN PRAKTIKUM.....	5
B. PENGANTAR LINKED LIST.....	8
C. SINGLY LINKED LIST.....	11
D. CIRCULAR LINKED LIST.....	20
E. DOUBLY LINKED LISTS.....	27
F. CIRCULAR DOUBLY LINKED LISTS.....	38
REFERENSI.....	50

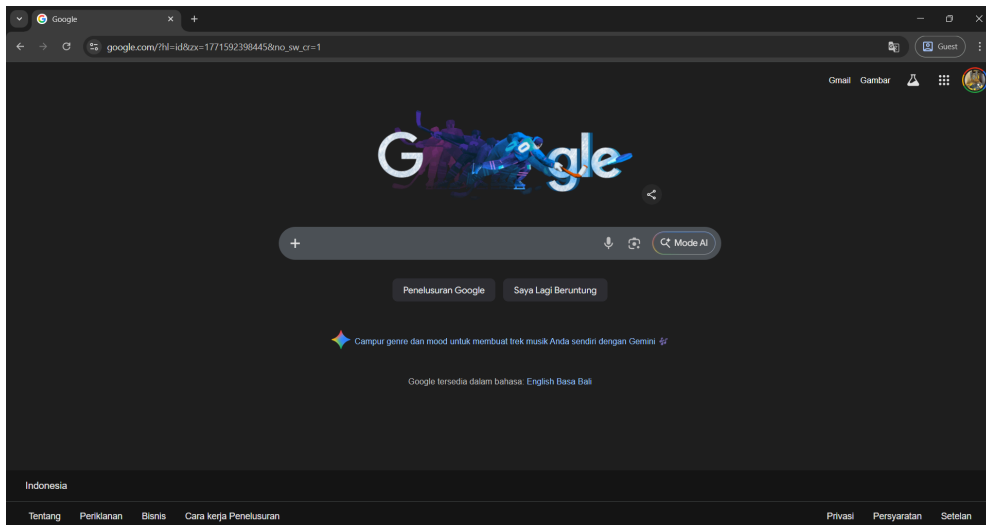
LINKED LIST

A. PERSIAPAN PRAKTIKUM

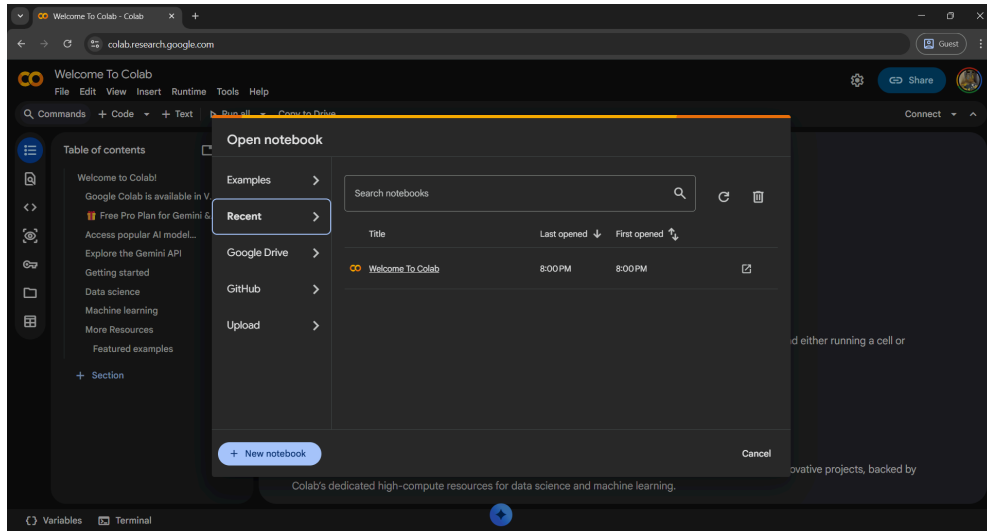
Karena praktikum ini menggunakan bahasa pemrograman Python, kita akan menggunakan Google Colaboratory (Colab) sebagai lembar kerja berbasis cloud. Mari kita siapkan lembar kerja terlebih dahulu.

Langkah Kerja:

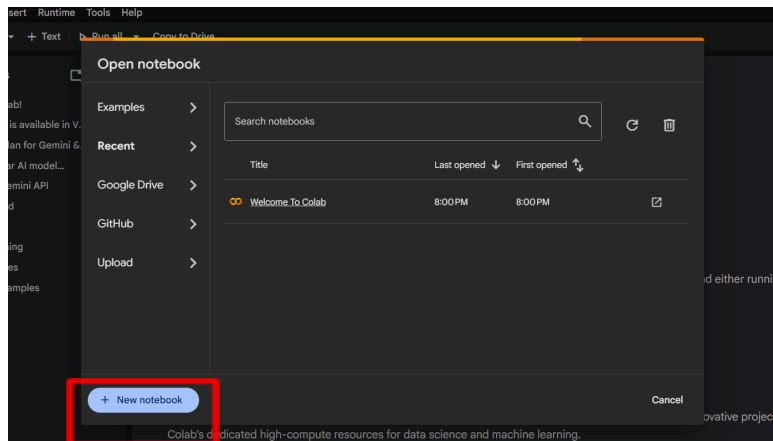
- 1) Buka Google Chrome atau browser lainnya, lalu pastikan Anda sudah login menggunakan akun Google (Gmail) masing-masing, serta terhubung ke internet.



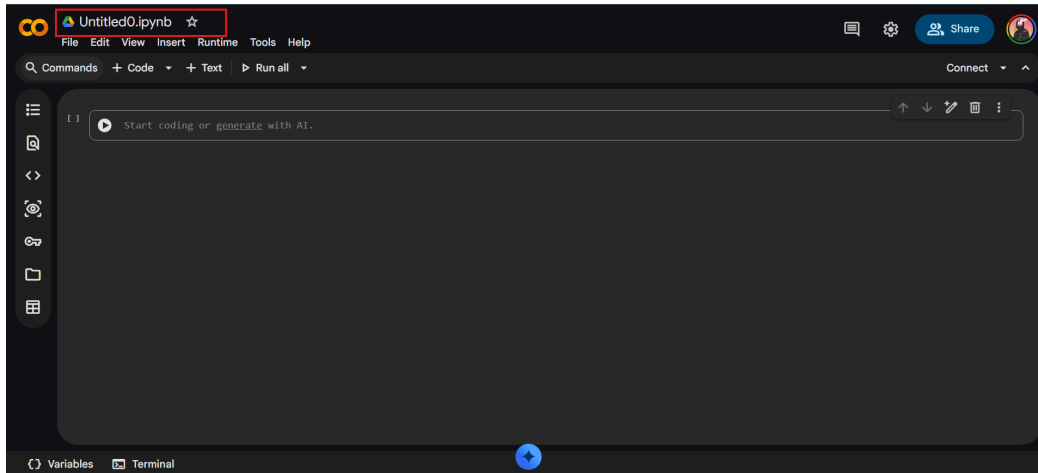
- 2) Kunjungi tautan berikut: <https://colab.research.google.com>



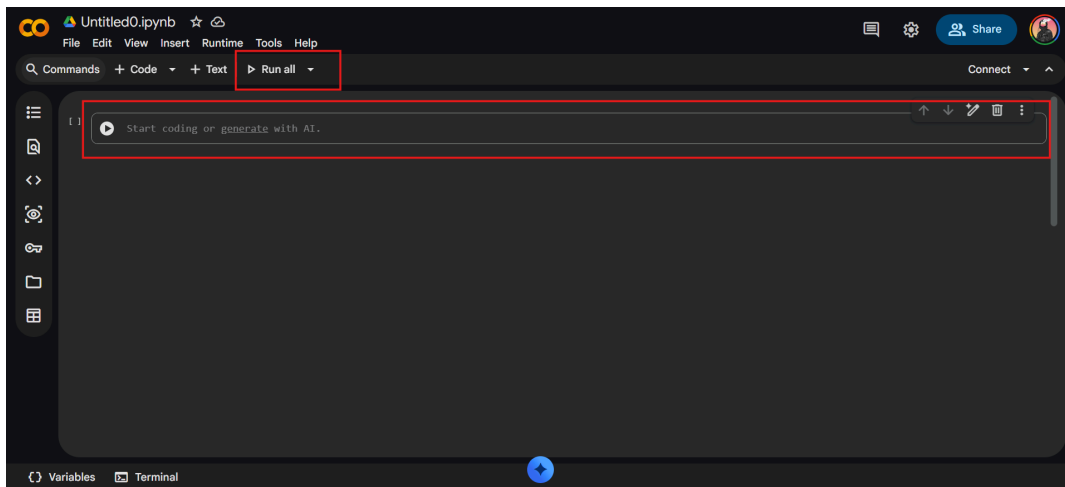
- 3) Hal yang pertama kita lakukan adalah membuat file baru. Pada jendela yang muncul, klik tombol New Notebook.



- 4) Di pojok kiri atas, klik nama file bawaan (biasanya Untitled0.ipynb) dan ubah formatnya menjadi **NIM_NamaLengkap_Modul2.ipynb**.



- 5) Di layar Anda sekarang terdapat sebuah kotak teks berlatar abu-abu. Ini disebut sebagai Cell. Di sinilah Anda akan mengetikkan kode Python. Untuk menjalankan kode, Anda cukup menekan ikon Play di sebelah kiri *cell*, atau menekan **Shift + Enter** di *keyboard*.



B. PENGANTAR LINKED LIST

1. Pengertian Linked List

Linked List adalah struktur data linear yang terdiri dari sekumpulan objek yang disebut Node. Berbeda dengan Array yang menyimpan data dalam blok memori yang berdampingan (seperti deretan kursi di bioskop), Linked List menyimpan data secara terpencar di memori.

Setiap Node minimal memiliki dua bagian:

- a. Data: Nilai atau informasi yang ingin disimpan.
- b. Pointer (Next): "Alamat" atau petunjuk yang mengarah ke node berikutnya.

Analoginya seperti sebuah rantai: setiap mata rantai (node) menyimpan informasi dan juga mengetahui di mana letak mata rantai berikutnya. Node pertama disebut Head, dan penunjuk pada node terakhir bernilai None (kosong), menandakan akhir dari list.

Perbandingan Array dan Linked List

Aspek	Array	Linked List
Alokasi Memori	Kontigu (Bersebelahan)	Tersebar (non-kontigu)
Ukuran	Tetap (statis)	Dinamis (fleksibel)
Akses Elemen	Langsung via indeks $O(1)$	Harus ditelusuri $O(n)$
Penyisipan/Penghapusan	Lambat, butuh pergeseran $O(n)$	Cepat, cukup ubah pointer $O(1)$
Penggunaan Memori	Lebih efisien (tanpa pointer)	Lebih besar (menyimpan pointer)

2. Implementasi class Node di Python (menggantikan pointer dan struct C)

Dalam bahasa C, sebuah elemen *Linked List* (disebut *node*) dibuat menggunakan struct tingkat rendah dan *pointer* memori manual. Karena Python tidak memiliki fitur tersebut, pendekatannya ditransformasikan menggunakan Pemrograman Berorientasi Objek (OOP) melalui pembuatan class Node.

Berikut adalah ringkasan jelas mengenai transformasinya:

a. Struct menjadi class

Di Python, *node* dibuat sebagai sebuah objek dari class *Node*. Kelas ini memiliki atribut *self.data* untuk menyimpan nilai, dan *self.next* untuk menautkan *node*.

b. Pointer menjadi Referensi Objek

Python tidak menggunakan *pointer* memori fisik (*next di C), melainkan menggunakan referensi objek (*self.next*) untuk menghubungkan antar *node* secara berurutan.

c. NULL menjadi None

Pada C, akhir sebuah antrean ditandai dengan *pointer* NULL (atau -1). Di Python, penanda batas akhir antrean yang kosong ini diganti dengan nilai *None*.

d. Alokasi Manual menjadi Garbage Collection

Di C, programmer wajib memesan dan membebaskan memori antrean secara manual. Di Python, hal ini dikelola sepenuhnya secara otomatis oleh Garbage Collection, di mana *node* yang dihapus dan putus referensinya akan langsung dibersihkan sistem dari memori tanpa perintah tambahan.

Ringkasnya, class *Node* di Python menyederhanakan logika *Linked List* C dengan menyembunyikan kerumitan manajemen memori dan pointer, menjadikannya jauh lebih aman dan ramah bagi pemula.

3. None sebagai Penanda Akhir List (Pengganti NULL)

Dalam konsep dasar *Linked List*, kita perlu mengetahui kapan sebuah antrean berakhir agar proses penelusuran (*traversing*) tidak terjadi terus-menerus.

- **Konsep di C:** Pada bahasa C, *node* terakhir dalam sebuah list akan menyimpan nilai spesial yang disebut NULL (atau terkadang direpresentasikan dengan -1) pada penunjuk (*pointer*) next-nya. Nilai NULL ini memberi tahu program bahwa tidak ada lagi alamat memori yang ditunjuk.
- **Transformasi di Python:** Python tidak mengenali konsep NULL maupun *pointer* memori mentah. Sebagai gantinya, Python menggunakan tipe data bawaan bernama

None. None adalah sebuah objek khusus di Python yang berarti "ketiadaan nilai" atau "kosong".

- **Penerapan:** Saat Anda membuat *node* paling akhir di sebuah *Singly* atau *Doubly Linked List*, Anda cukup mengatur atribut rujukan ke depannya menjadi kosong. Contohnya: `self.next = None`. Saat melakukan *looping* untuk membaca data, kondisi berhentinya adalah ketika penelusuran sudah bertemu dengan None.

Contoh implementasi kondisi batas None pada class node:

```
class Node:
    def __init__(self, data):
        self.data = data    # Menyimpan nilai
        self.next = None    # Penanda akhir list (pengganti NULL)

# Membuat beberapa node
node1 = Node(10)
node2 = Node(20)
node3 = Node(30)

# Menghubungkan node secara manual
node1.next = node2    # node1 -> node2
node2.next = node3    # node2 -> node3
# node3.next tetap None -> menandakan akhir list
```

4. *Garbage Collection* Otomatis di Python (Pengganti malloc dan free)

Perbedaan terbesar dan paling memudahkan pemula saat menggunakan Python dibandingkan C adalah cara mengelola memorinya.

- **Konsep di C (Alokasi Manual):** Di C, *programmer* harus mencari sendiri ruang memori kosong di komputer (disebut *free pool* atau AVAIL). Saat menambah *node*, Anda harus memesan memori secara manual menggunakan perintah seperti `malloc()`. Sebaliknya, saat menghapus *node*, Anda wajib mengosongkan memori tersebut secara manual menggunakan perintah `free()` agar memori bisa digunakan kembali oleh program lain. Jika Anda lupa melakukan `free()`, akan terjadi *memory leak* (kebocoran memori).

- **Transformasi di Python (Garbage Collector):** Python memiliki sistem otomatis di balik layar yang bertugas membersihkan memori, disebut Garbage Collector. Saat Anda memutus referensi sebuah node (mengubah pointer agar tidak menunjuk ke node tersebut), Python akan secara otomatis membebaskan ruang memorinya.
- **Penerapan saat Menghapus Node:** Di Python, saat Anda ingin menghapus sebuah *node* dari antrean, Anda tidak perlu memanggil fungsi untuk menghancurkan memori objek tersebut. Anda cukup memutuskan atau mengalihkan referensi sambungannya (mengubah next atau prev pada *node* di sekitarnya agar tidak menunjuk ke *node* target lagi).

Ketika *Garbage Collector* menyadari ada sebuah objek *node* yang "melayang" dan tidak ada satu pun variabel yang merujuk/menunjuk kepadanya, sistem Python akan menganggap objek tersebut sebagai sampah (*garbage*) dan secara otomatis membebaskan ruang memorinya kembali ke sistem komputer

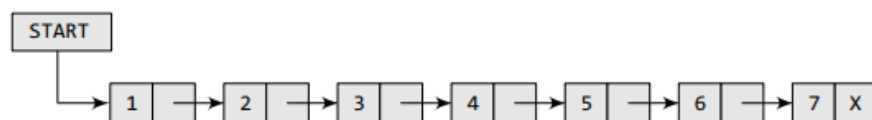
Contoh Singkat Logika Penghapusan di Python:

```
# Misalnya kita ingin menghapus node kedua
# Kita cukup menyambungkan node pertama (ptr) LANGSUNG ke node ketiga
(ptr.next.next)
ptr.next = ptr.next.next

# Node kedua sekarang "putus" dari antrean.
# Garbage Collector Python akan otomatis menghapusnya dari memori
```

C. SINGLY LINKED LIST

Singly Linked List (Linked List Tunggal) adalah jenis linked list yang paling sederhana. Setiap node hanya memiliki satu penunjuk yaitu next, yang mengarah ke node berikutnya. Penelusuran hanya bisa dilakukan satu arah: dari Head menuju akhir list.



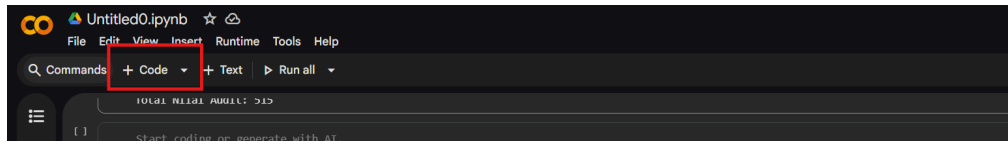
Gambar 1. Singly Linked List

1. Inisialisasi

Sebelum melakukan operasi apapun pada Singly Linked List, kita perlu membangun fondasi strukturnya terlebih dahulu. Inisialisasi dilakukan dengan mendefinisikan dua buah class, yaitu class `Node` sebagai cetak biru setiap elemen, dan class `SinglyLinkedList` sebagai pengontrol seluruh antrian. Terdapat tiga komponen utama yang perlu dipahami:

- **Class `Node`:** Merepresentasikan satu elemen dalam list. Memiliki dua atribut yaitu *`self.data`* untuk menyimpan nilai, dan *`self.next`* untuk menunjuk ke node berikutnya.
- **Class `SinglyLinkedList`:** Berperan sebagai manajer terpusat yang menyimpan variabel *`self.start`* (HEAD), yaitu titik awal penelusuran list.
- **Aturan Batas:** Atribut *`self.next`* pada node terakhir selalu bernilai `None`, sebagai penanda bahwa tidak ada node sesudahnya. Tanpa penanda ini, perulangan tidak akan mengetahui kapan harus berhenti.

Silahkan buat 1 cell dan coba ketikkan kode ini di Google Colab :



```
# Template Singly Linked List
class Node:
    def __init__(self, data):
        self.data = data # Menyimpan nilai
        self.next = None # Penunjuk ke node berikutnya (awalnya kosong)

class SinglyLinkedList:
    def __init__(self):
        self.start = None # Variabel Head / START (list awalnya kosong)
```

CATATAN PENTING: Aturan Indentasi Python pada Class

Bahasa pemrograman Python sangat ketat terhadap indentasi (jarak spasi di awal baris). Saat Anda diminta untuk menambahkan fungsi baru (misalnya `def display(self):`) ke dalam sebuah class, pastikan penulisan `def` menjorok ke dalam (menggunakan satu kali *Tab*) agar sejajar dengan

fungsi `def __init__(self):` di atasnya.

Jika baris `def` menempel rata di ujung kiri layar, Python akan menganggapnya sebagai fungsi eksternal biasa, bukan sebagai bagian dari *class* tersebut, yang akan menyebabkan *Error* saat program dijalankan.

2. Traversing & Displaying (Penelusuran dan Penampilan Data)

Traversing adalah proses mengunjungi setiap node dalam linked list satu per satu menggunakan perulangan (loop). Kita menggunakan variabel penunjuk sementara (ptr) agar variabel HEAD/start tidak bergeser dari posisi awalnya.

Logika Traversing:

- 1) Atur `ptr = self.start` (mulai dari HEAD).
- 2) Selama `ptr` bukan `None`, baca data pada `ptr`.
- 3) Geser `ptr` ke node berikutnya: `ptr = ptr.next`.
- 4) Berhenti saat `ptr == None` (akhir list).

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Menampilkan seluruh isi Singly Linked List
def display(self):
    if self.start is None:
        print('List Kosong')
        return

    ptr = self.start # Mulai dari HEAD
    print('Isi List: ', end='')
    while ptr is not None:
        print(f'[{ptr.data}]', end=' -> ')
        ptr = ptr.next # Geser ke node berikutnya
    print('None')
```

3. Searching (Pencarian Data)

Searching adalah proses mencari node dengan nilai tertentu di dalam list. Karena Singly Linked List tidak memiliki indeks seperti array, pencarian dilakukan secara linear dari HEAD hingga akhir list.

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Mencari data dalam Singly Linked List
def search(self, val):
    ptr = self.start
    posisi = 1 # Melacak urutan node

    while ptr is not None:
        if ptr.data == val:
            print(f'Data {val} ditemukan pada urutan ke-{posisi}.')
            return ptr
        ptr = ptr.next
        posisi += 1

    print(f'Data {val} tidak ditemukan di dalam list.')
    return None
```

4. Inserting (Penyisipan Node Baru)

Penyisipan elemen baru mengharuskan kita untuk memperbarui referensi (pointer) agar rantai list tidak terputus. Terdapat 4 kondisi utama penyisipan:

a. Penyisipan di Awal List

Node baru disambungkan ke node HEAD lama, lalu variabel start digeser ke node baru.

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Insert di awal list
def insert_beg(self, val):
    new_node = Node(val)          # Buat node baru
    new_node.next = self.start    # Hubungkan ke HEAD lama
    self.start = new_node         # Geser HEAD ke node baru
```

b. Penyisipan di Akhir List

Kita menelusuri list hingga menemukan node terakhir (yang next-nya bernilai None), lalu menautkan node baru di posisi tersebut.

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Insert di akhir list
def insert_end(self, val):
    new_node = Node(val)
    if self.start is None:      # Kasus list kosong
        self.start = new_node
        return

    ptr = self.start
    while ptr.next is not None: # Cari node terakhir
        ptr = ptr.next
    ptr.next = new_node        # Sambungkan node baru di akhir
```

c. Penyisipan Setelah Node Spesifik

Kita mencari node dengan nilai target tertentu, lalu menyisipkan node baru tepat setelah node tersebut.

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Insert setelah node spesifik (berdasarkan nilainya)
def insert_after(self, target_val, val):
    ptr = self.start
    while ptr is not None and ptr.data != target_val:
        ptr = ptr.next

    if ptr is not None:      # Jika target ditemukan
        new_node = Node(val)
        new_node.next = ptr.next # Node baru menunjuk ke node setelah
target
        ptr.next = new_node    # Target menunjuk ke node baru
    else:
        print('Target node tidak ditemukan!')
```

d. Penyisipan Sebelum Node Spesifik

Kita mencari node sebelum node target, lalu menyisipkan node baru di antara keduanya.

Diperlukan variabel penunjuk sebelumnya (prev) untuk melacak node di belakang ptr.

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Insert sebelum node spesifik (berdasarkan nilainya)
def insert_before(self, target_val, val):
    if self.start is None:
        print('List Kosong')
        return

    # Kasus khusus: target ada di HEAD
    if self.start.data == target_val:
        self.insert_beg(val)
        return

    prev = None
    ptr = self.start
    while ptr is not None and ptr.data != target_val:
        prev = ptr
        ptr = ptr.next

    if ptr is not None:          # Jika target ditemukan
        new_node = Node(val)
        new_node.next = ptr      # Node baru menunjuk ke target
        prev.next = new_node     # Node sebelumnya menunjuk ke node
baru
    else:
        print('Target node tidak ditemukan!')
```

5. Deleting (Penghapusan Node)

Penghapusan node mengharuskan kita memutus referensi ke node yang dihapus agar Garbage Collector Python dapat membersihkan memorinya. Terdapat 3 kondisi utama penghapusan:

a. Penghapusan di Awal List

Variabel start cukup digeser ke node kedua. Node pertama otomatis kehilangan referensi dan akan dihapus oleh Garbage Collector.

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Delete di awal list
def delete_beg(self):
    if self.start is None:
        print('List Kosong (Underflow)')
        return
    print(f'Node {self.start.data} telah dihapus.')
    self.start = self.start.next # Geser HEAD ke node kedua
```

b. Penghapusan di Akhir List

Kita menelusuri hingga menemukan node sebelum terakhir, lalu mengubah next-nya menjadi None sehingga node terakhir terputus dari list.

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Delete di akhir list
def delete_end(self):
    if self.start is None:
        print('List Kosong (Underflow)')
        return

    # Kasus khusus: hanya ada satu node
    if self.start.next is None:
        print(f'Node {self.start.data} telah dihapus.')
        self.start = None
        return

    ptr = self.start
    while ptr.next.next is not None: # Berhenti di node sebelum
    terakhir
        ptr = ptr.next
    print(f'Node {ptr.next.data} telah dihapus.')
    ptr.next = None # Putuskan referensi ke node terakhir
```

c. Penghapusan Setelah Node Spesifik

Kita mencari node target, lalu meloncati node setelahnya dengan langsung menyambungkan ke node dua langkah setelah target.

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
# Delete setelah node spesifik
def delete_after(self, target_val):
    ptr = self.start
    while ptr is not None and ptr.data != target_val:
        ptr = ptr.next

    if ptr is not None and ptr.next is not None:
        print(f'Node {ptr.next.data} (setelah {target_val}) telah dihapus.')
        ptr.next = ptr.next.next # Lompati node setelah target
    elif ptr is None:
        print('Target tidak ditemukan!')
    else:
        print('Tidak ada node setelah target!')
```

6. Eksekusi Program

Setelah kita mendefinisikan `class Node` dan `class SinglyLinkedList` beserta seluruh metode operasinya (inisialisasi, traversing, inserting, searching, dan deleting), sistem belum akan menghasilkan apa-apa karena kita baru membuat "cetak biru" dari struktur datanya. Kita perlu membuat blok eksekusi utama.

Dalam bahasa C, eksekusi program biasanya dibungkus di dalam fungsi `main()` menggunakan antarmuka menu dengan perulangan `do-while`. Saat ditransformasikan ke Python (khususnya di lingkungan Google Colab), kita menyederhanakannya menjadi sekumpulan perintah pemanggilan metode secara langsung yang dieksekusi dari atas ke bawah. Pendekatan ini jauh lebih interaktif untuk keperluan pengujian.

Silakan tambahkan blok kode ini di baris paling bawah pada sel Google Colab Anda, tepat di luar indentasi `class` yang telah didefinisikan sebelumnya:

```

# --- EKSEKUSI PROGRAM ---
# 1. Inisialisasi list kosong
my_list = SinglyLinkedList()

# 2. Tambah data (Insert)
print("--- Proses Insert ---")
my_list.insert_beg(10)
my_list.insert_end(20)
my_list.insert_beg(5)
my_list.insert_after(10, 15)
my_list.display()

# 3. Cari data (Search)
print("\n--- Proses Search ---")
my_list.search(15)

# 4. Hapus data (Delete)
print("\n--- Proses Delete ---")
my_list.delete_beg()
my_list.delete_end()
my_list.display()

# 5. Cek setelah manipulasi
print("\n--- Hasil Akhir ---")
my_list.display()

```

Output:

```

... --- Proses Insert ---
Isi List: [5] -> [10] -> [15] -> [20] -> None

--- Proses Search ---
Data 15 ditemukan pada urutan ke-3.

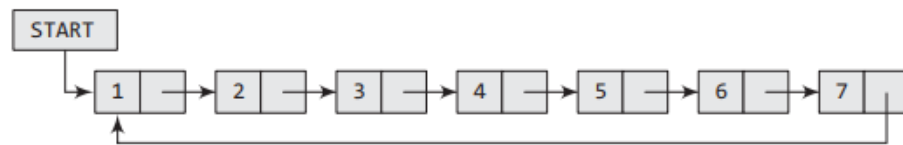
--- Proses Delete ---
Node 5 telah dihapus.
Node 20 telah dihapus.
Isi List: [10] -> [15] -> None

--- Hasil Akhir ---
Isi List: [10] -> [15] -> None

```

D. CIRCULAR LINKED LIST

Circular Linked List (Linked List Melingkar) adalah variasi dari struktur data *linked list* dimana *node* paling akhir menyimpan referensi (penunjuk) yang mengarah kembali ke *node* pertama (START atau Head). Karena bentuknya yang mengunci membentuk cincin tertutup, struktur ini tidak memiliki awalan dan akhiran yang mutlak, serta tidak ada penunjuk referensi yang bernilai None (pengganti NULL di C) di sepanjang antriannya.

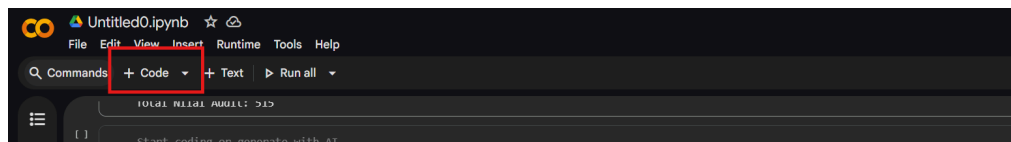


Gambar 2. Circular Linked List

1. Inisialisasi

Di Python, kita menggunakan `class Node` untuk menggantikan `struct C`. Inisialisasi list melingkar dimulai dengan mengatur `START` menjadi `None`. Saat *node* pertama ditambahkan, penunjuk `next` dari *node* tersebut akan langsung dihubungkan ke dirinya sendiri untuk membentuk lingkaran

silahkan buat 1 cell dan coba ketikkan kode ini di google colab :



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class CircularLinkedList:
    def __init__(self):
        self.start = None # Inisialisasi list kosong
```

2. Traversing (Penelusuran)

Penelusuran digunakan untuk mengunjungi setiap elemen. Karena tidak ada nilai `None` di akhir list, perulangan (seperti simulasi `do-while` di C) akan dihentikan ketika penunjuk telah berputar dan kembali merujuk pada *node* START

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas `Node` :

```
def display(self):
    if self.start is None:
        print("List kosong.")
        return

    ptr = self.start
    print("Isi Circular Linked List: ", end="")
    while True:
        print(f"[{ptr.data}]", end=" -> ")
        ptr = ptr.next
        # Berhenti jika sudah kembali ke node pertama
        if ptr == self.start:
            break
    print("(Kembali ke START)")
```

3. Inserting (Menyisipkan Node)

Penyisipan pada struktur melingkar membutuhkan perhatian khusus. Setiap kali kita mengubah *node* pertama (di awal) atau *node* terakhir (di akhir), kita harus memastikan tautan lingkarannya tetap tertutup rapat.

a. Penyisipan di awal list

Node baru disambungkan ke `START` lama, lalu iterasi mencari *node* paling akhir untuk menautkan ujung lingkarannya ke *node* baru tersebut. Terakhir, *node* baru dijadikan `START`

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas `Node` :

```

def insert_beg(self, val):
    new_node = Node(val)
    if self.start is None:
        self.start = new_node
        new_node.next = self.start # Menunjuk ke diri sendiri
    else:
        ptr = self.start
        # Cari node terakhir
        while ptr.next != self.start:
            ptr = ptr.next

        new_node.next = self.start # Node baru menunjuk START
lama
        ptr.next = new_node        # Node terakhir menunjuk node
baru
        self.start = new_node      # Node baru resmi menjadi
START

```

b. Penyisipan di akhir list

Iterasi mencari *node* paling akhir, lalu menautkannya ke *node* baru. *Node* baru kemudian ditautkan kembali ke START

Tambahkan code di bawah ini dalam cell yang sama, dibawah method *insert_beg*:

```

def insert_end(self, val):
    new_node = Node(val)
    if self.start is None:
        self.insert_beg(val)
        return

    ptr = self.start
    # Cari node terakhir
    while ptr.next != self.start:
        ptr = ptr.next

```

```
ptr.next = new_node      # Node terakhir lama menunjuk node
baru
new_node.next = self.start # Node baru mengunci lingkaran ke
START
```

4. Searching (Pencarian)

Pencarian beroperasi dengan mencocokkan data pada *node*. Sama seperti operasi *traversing*, kondisi berhentinya adalah ketika penunjuk sudah berkeliling satu putaran penuh dan kembali ke titik START.

Tambahkan code di bawah ini dalam cell yang sama, dibawah method *insert_end* :

```
def search(self, val):
    if self.start is None:
        return None

    ptr = self.start
    posisi = 1
    while True:
        if ptr.data == val:
            print(f>Data {val} ditemukan di posisi ke-{posisi}.")
            return ptr
        ptr = ptr.next
        posisi += 1
        if ptr == self.start: # Jika sudah kembali ke awal
            break

    print(f>Data {val} tidak ditemukan.")
    return None
```

5. Deleting (Menghapus Node)

Penghapusan *node* dilakukan dengan memotong referensi (*next*) dan membiarkan *Garbage Collection* otomatis Python menghapus memori *node* tersebut.

a. Penghapusan di awal list

Cari *node* terakhir, tautkan *node* terakhir langsung memotong ke *node* ke-2 (*start.next*), lalu geser *START* ke *node* ke-2

Tambahkan code di bawah ini dalam cell yang sama, dibawah method *search* :

```
def delete_beg(self):
    if self.start is None:
        return
    ptr = self.start
    if ptr.next == self.start: # Jika hanya tersisa 1 node
        self.start = None
    else:
        # Cari node terakhir
        while ptr.next != self.start:
            ptr = ptr.next

        ptr.next = self.start.next # Lingkaran melompati node
        pertama
        self.start = self.start.next # Pindahkan START ke node
        kedua
```

b. Penghapusan di akhir list

Iterasi menggunakan dua penunjuk (*ptr* dan *preptr*) untuk mencari *node* terakhir dan *node* sebelum terakhir. Tautkan *node* sebelum terakhir (*preptr*) langsung ke *START*

Tambahkan code di bawah ini dalam cell yang sama, dibawah method *delete_beg*:

```
def delete_end(self):
    if self.start is None:
        return
```



```

ptr = self.start
if ptr.next == self.start: # Jika hanya tersisa 1 node
    self.start = None
else:
    # Cari node terakhir (ptr) dan node sebelum terakhir (preptr)
    while ptr.next != self.start:
        preptr = ptr
        ptr = ptr.next

    preptr.next = self.start # Mengunci ulang lingkaran tanpa
node terakhir

```

6. Eksekusi Program

Sama halnya dengan pengujian pada Singly Linked List sebelumnya, kita akan melakukan instansiasi objek dan memanggil metode manipulasi secara sekuensial. Pemanggilan `display()` secara berkala sangat disarankan untuk memvalidasi apakah tautan cincin memori (Circular) terhubung dengan benar pasca-operasi penyisipan atau penghapusan.

Silakan tambahkan blok pengujian ini di sel terbawah Anda:

```

# 1. INSTANSIASI (Membuat objek list baru)
c11 = CircularLinkedList()

# 2. PROSES INSERTING
print("--- Proses Menambahkan Data ---")
c11.insert_beg(20) # Menambahkan 20 di awal
c11.insert_beg(10) # Menambahkan 10 di awal (sebelum 20)
c11.insert_end(30) # Menambahkan 30 di akhir
c11.insert_end(40) # Menambahkan 40 di akhir

# 3. PROSES TRAVERSING (Menampilkan List)
c11.display()
# Ekspektasi Output: [4] -> [5] -> [6] -> [7] -> (Kembali ke START)

```

```

# 4. PROSES SEARCHING
print("\n--- Proses Mencari Data ---")
c1l.search(30)    # Mencari data yang ada
c1l.search(100)   # Mencari data yang tidak ada

# 5. PROSES DELETING
print("\n--- Proses Menghapus Data ---")
c1l.delete_beg() # Menghapus node pertama (10)
c1l.delete_end() # Menghapus node terakhir (40)

# Menampilkan kembali isi list setelah penghapusan
print("Isi list setelah dihapus:")
c1l.display()
# Ekspektasi Output: [5] -> [6] -> (Kembali ke START)

```

Output :

```

*** --- Proses Menambahkan Data ---
Isi Circular Linked List: [10] -> [20] -> [30] -> [40] -> (Kembali ke START)

--- Proses Mencari Data ---
Data 30 ditemukan di posisi ke-3.
Data 100 tidak ditemukan.

--- Proses Menghapus Data ---
Isi list setelah dihapus:
Isi Circular Linked List: [20] -> [30] -> (Kembali ke START)

```

E. DOUBLY LINKED LISTS

Doubly Linked List atau *two-way linked list* adalah tipe linked list yang lebih kompleks di mana setiap simpul (node) menyimpan dua penunjuk (pointer/referensi), yaitu menuju ke node sesudahnya dan node sebelumnya.

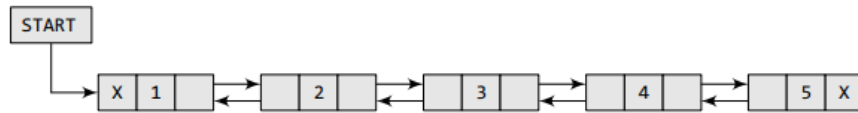


Figure 6.37 Doubly linked list

Struktur ini memiliki karakteristik utama:

- Dapat ditelusuri maju (*forward*) maupun mundur (*backward*).
- Membutuhkan ruang memori lebih besar per node-nya karena menyimpan dua referensi alamat.
- Memberikan kemudahan luar biasa dalam memanipulasi elemen (seperti menghapus node di tengah) karena kita memiliki akses langsung ke node sebelumnya tanpa harus menelusuri dari awal.

1. Inisialisasi

Dalam arsitektur Doubly Linked List, perakitan struktur memori dilakukan secara manual. Sebuah simpul (node) dirancang secara spesifik agar dapat merujuk ke dua arah referensi sekaligus. Untuk menginisialisasinya, diperlukan komponen-komponen berikut:

a. Komponen Simpul (Node)

Kita mentransformasikannya menjadi objek `class Node` yang memuat tiga atribut utama:

- `prev`: Menunjuk ke objek node sebelumnya.
- `data`: Menyimpan nilai atau konten data.
- `next`: Menunjuk ke objek node sesudahnya.

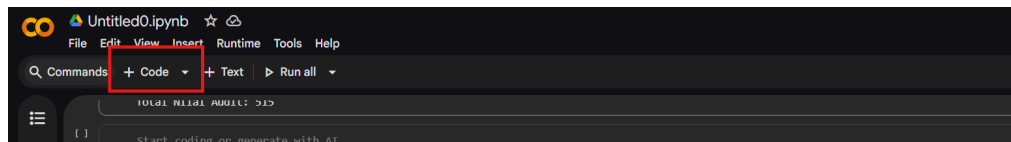
b. Komponen Pengontrol (DoublyLinkedList)

Variabel pada class list yang memegang kendali atas antrean. Atribut terpentingnya adalah `start`, yang bertugas melacak node pertama (Head).

c. Aturan Batas

Ujung referensi `prev` pada node pertama dan `next` pada node terakhir selalu bernilai `None` (sebagai pengganti NULL), yang menandakan ujung dari daftar tersebut.

Silahkan buat 1 cell dan coba ketikkan kode ini di Google Colab :



```
class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.start = None
```

2. Traversing

Dalam mekanisme penelusuran Doubly Linked List, akses data memerlukan iterasi struktural dari satu simpul ke simpul lainnya menggunakan variabel penunjuk sementara (`ptr`). Karena arsitektur ini memiliki dua penunjuk arah (`next` dan `prev`), terdapat dua cara untuk menelusuri struktur data ini, yaitu:

a. Menelusuri dari Awal hingga Akhir (Maju)

Metode ini memanfaatkan referensi `next` untuk bergerak maju. Iterasi dieksekusi secara berurutan dari simpul pertama (Head) hingga kursor menyentuh batas ruang kosong (`None`) di ujung akhir antrean.

Silakan tambahkan blok fungsi pertama ini ke dalam class `DoublyLinkedList` di Google Colab:

```
def display_maju(self):
    ptr = self.start
    print("Maju : ", end="")

    while ptr is not None:
        print(f"[{ptr.data}]", end=" <-> ")
        ptr = ptr.next
    print("None")
```

b. Menelusuri dari Akhir hingga Awal (Mundur)

Metode ini memanfaatkan referensi `prev` untuk bergerak mundur. Mengingat sistem hanya melacak posisi awal list (Head), proses ini memerlukan dua tahapan: iterasi pendahuluan untuk memposisikan kursor tepat di simpul paling akhir, dilanjutkan dengan iterasi penelusuran ke arah simpul pertama.

Selanjutnya, tambahkan blok fungsi kedua ini di bawah fungsi sebelumnya:

```
def display_mundur(self):
    ptr = self.start
    if ptr is None:
        print("List Kosong")
        return

    # Tahap 1: Iterasi pendahuluan untuk memposisikan kursor di
    # simpul paling akhir
    # Evaluasi berhenti tepat di node terakhir, bukan di batas
    None
    while ptr.next is not None:
        ptr = ptr.next

    # Tahap 2: Menjalankan penelusuran mundur memanfaatkan
    # referensi prev
    print("Mundur : ", end="")
    while ptr is not None:
        print(f"[{ptr.data}]", end=" <-> ")
```

```
ptr = ptr.prev  
print("None")
```

3. Searching

Secara teoretis, arsitektur Doubly Linked List menawarkan potensi efisiensi pencarian yang tinggi. Apabila sistem menyimpan referensi simpul paling akhir (Tail), penelusuran data dapat dioptimalkan dengan mengevaluasi dari dua arah secara bersamaan. Namun, pada implementasi standar yang hanya mengandalkan titik awal (Head), operasi pencarian dilakukan melalui penelusuran maju secara linear.

Mekanisme ini bekerja dengan mengevaluasi atribut **data** pada setiap simpul selama iterasi berlangsung. Variabel penghitung (*counter*) juga ditambahkan untuk melacak posisi logis elemen di dalam antrean memori.

Silakan tambahkan fungsi operasi pencarian ini ke dalam class `DoublyLinkedList` di Google Colab Anda:

```
def search(self, val):  
    ptr = self.start  
    posisi = 1    # Variabel untuk melacak urutan simpul  
  
    while ptr is not None:  
        if ptr.data == val:  
            print(f>Data {val} ditemukan pada urutan ke-{posisi}.")  
            return ptr  
  
        ptr = ptr.next  
        posisi += 1  
  
    print(f>Data {val} tidak ditemukan di dalam list.")  
    return None
```

4. Inserting

Penyisipan elemen pada struktur dua arah membutuhkan pembaruan tautan secara simultan. Terdapat empat kondisi utama operasi penyisipan yang wajib diimplementasikan:

a. Penyisipan di Awal Antrian

Metode ini mengeksekusi penambahan simpul baru tepat sebelum simpul pertama (Head). Operasi ini menuntut pembaruan referensi `prev` pada simpul awal yang lama dan penggeseran variabel pengontrol `start` ke objek simpul yang baru.

Silakan tambahkan blok fungsi pertama ini ke dalam `class DoublyLinkedList` di Google Colab Anda:

```
def insert_beg(self, val):
    new_node = Node(val) # Menginisialisasi objek simpul baru

    # Evaluasi jika antrean masih kosong
    if self.start is None:
        self.start = new_node
    else:
        # Menautkan referensi ganda di titik awal
        new_node.next = self.start
        self.start.prev = new_node
        self.start = new_node # Menggeser Head ke simpul baru
```

b. Penyisipan di Akhir Antrean

Metode ini menambahkan simpul baru di ujung struktur. Karena sistem hanya melacak titik awal (Head), diperlukan iterasi pendahuluan menggunakan variabel penunjuk sementara untuk menemukan simpul paling akhir sebelum menautkan referensi ke simpul baru.

Selanjutnya, tambahkan blok fungsi kedua ini di bawah fungsi sebelumnya:

```
def insert_end(self, val):
    new_node = Node(val)

    # Evaluasi jika antrean masih kosong
    if self.start is None:
        self.start = new_node
        return

    ptr = self.start
```

```

# Iterasi mencari simpul paling akhir
while ptr.next is not None:
    ptr = ptr.next

# Menautkan referensi ganda di batas akhir
ptr.next = new_node
new_node.prev = ptr

```

c. Penyisipan Setelah Target Spesifik

Metode ini memadukan mekanisme pencarian dan penyisipan. Setelah iterasi berhasil menemukan simpul target, operasi penautan dilakukan untuk menyisipkan simpul baru persis di antara target tersebut dan simpul sesudahnya.

Lanjutkan dengan menambahkan blok fungsi ketiga ini:

```

def insert_after(self, target_val, val):
    new_node = Node(val)
    ptr = self.start

    # Iterasi pencarian simpul target
    while ptr is not None and ptr.data != target_val:
        ptr = ptr.next

    # Eksekusi jika target ditemukan
    if ptr is not None:
        new_node.prev = ptr
        new_node.next = ptr.next

    # Evaluasi keamanan jika simpul target bukan simpul
penutup
        if ptr.next is not None:
            ptr.next.prev = new_node
            ptr.next = new_node
        else:
            print("Target node tidak ditemukan!")

```

d. Penyisipan Sebelum Target Spesifik

Serupa dengan metode ketiga, namun penautan memori dilakukan tepat sebelum simpul target. Kondisi ini memerlukan evaluasi keamanan ekstra jika target yang dicari kebetulan adalah simpul pertama (Head).

Terakhir, tambahkan blok fungsi keempat ini untuk melengkapi operasi penyisipan:

```
def insert_before(self, target_val, val):
    new_node = Node(val)
    ptr = self.start

    # Iterasi pencarian simpul target
    while ptr is not None and ptr.data != target_val:
        ptr = ptr.next

    # Eksekusi jika target ditemukan
    if ptr is not None:
        new_node.next = ptr
        new_node.prev = ptr.prev

        # Evaluasi keamanan jika simpul target bukan simpul pertama
        if ptr.prev is not None:
            ptr.prev.next = new_node
        else:
            self.start = new_node # Menggeser Head jika target adalah
simpul pertama
        ptr.prev = new_node
    else:
        print("Target node tidak ditemukan!")
```

5. Deleting

Penghapusan elemen menuntut pemutusan dan penyambungan ulang referensi ganda agar memori simpul yang terisolasi dapat dibersihkan oleh sistem *Garbage Collection* Python. Terdapat tiga kondisi operasi penghapusan yang umum diimplementasikan:

a. Penghapusan di Awal Antrean

Metode ini bertugas melepaskan simpul pertama (Head). Operasi ini menggeser variabel pengontrol `start` ke simpul kedua, lalu memutuskan referensi mundur (`prev`) dari simpul kedua tersebut agar tidak lagi menunjuk ke simpul pertama yang akan dibuang.

Silakan tambahkan blok fungsi pertama ini ke dalam class `DoublyLinkedList` di Google Colab Anda:

```
def delete_beg(self):
    # Mencegah penghapusan pada list kosong (Underflow)
    if self.start is None:
        print("List Kosong, tidak ada elemen yang dapat
dihapus.")
        return

    ptr = self.start
    self.start = self.start.next

    # Jika antrean memiliki lebih dari satu simpul
    if self.start is not None:
        self.start.prev = None # Memutus referensi mundur ke
simpul lama

    print(f"Simpul dengan nilai {ptr.data} di awal list telah
dihapus.")
```

b. Penghapusan di Akhir Antrean

Metode ini mengeksekusi pelepasan simpul paling akhir (Tail). Sistem diwajibkan melakukan iterasi penelusuran untuk mengidentifikasi simpul terakhir, lalu memutus referensi maju (`next`) dari simpul tepat di belakangnya menjadi `None`.

Selanjutnya, tambahkan blok fungsi kedua ini di bawah fungsi sebelumnya:

```
def delete_end(self):
    if self.start is None:
        print("List Kosong, tidak ada elemen yang dapat
dihapus.")
        return

    ptr = self.start
    # Evaluasi khusus jika antrean hanya memiliki tepat satu
```

```

simpul
    if ptr.next is None:
        self.start = None
        print(f"Simpul tunggal dengan nilai {ptr.data} telah
dihapus.")
        return

    while ptr.next is not None:
        ptr = ptr.next

    # Memutus referensi dari simpul sebelum simpul terakhir
    ptr.prev.next = None
    print(f"Simpul dengan nilai {ptr.data} di akhir list telah
dihapus.")

```

c. Penghapusan Target Spesifik

Metode ini mengintegrasikan mekanisme pencarian dan penghapusan sekaligus. Kursor akan mencari simpul dengan nilai data tertentu, lalu menjahit tautan referensi antara simpul yang berada tepat sebelum dan sesudah simpul target tersebut.

Terakhir, tambahkan blok fungsi ketiga ini untuk melengkapi operasi manipulasi memori:

```

def delete_node(self, target_val):
    if self.start is None:
        print("List Kosong.")
        return

    ptr = self.start
    # Iterasi pencarian simpul target
    while ptr is not None and ptr.data != target_val:
        ptr = ptr.next

    # Evaluasi jika iterasi selesai namun target tidak ditemukan
    if ptr is None:
        print("Target simpul tidak ditemukan di dalam list.")
        return

    # Kondisi khusus: Jika target yang ditemukan adalah simpul pertama
    if ptr == self.start:
        self.start = ptr.next

```

```

        if self.start is not None:
            self.start.prev = None
        # Kondisi normal: Target berada di tengah atau di akhir
    else:
        ptr.prev.next = ptr.next
        # Evaluasi keamanan jika target bukan simpul penutup (akhir)
        if ptr.next is not None:
            ptr.next.prev = ptr.prev

    print(f"Simpul target dengan nilai {target_val} telah berhasil
    dihapus.")

```

6. Eksekusi Program

Sama halnya dengan pengujian arsitektur sebelumnya, mari kita lakukan instansiasi objek `DoublyLinkedList` dan menjalankan operasi manipulasi memori secara sekuensial guna menguji integritas fungsi referensi ganda (*prev* dan *next*).

Silakan tambahkan blok pengujian ini di sel terbawah Google Colab Anda, tepat di luar area indentasi *class*:

```

# 1. Inisialisasi object memori
dll = DoublyLinkedList()

# 2. Masukkan data di awal
dll.insert_beg(10) # List: 10
dll.insert_beg(5)  # List: 5 <-> 10
print("Setelah insert_beg:")
dll.display_maju()

# 3. Masukkan data di akhir
dll.insert_end(20) # List: 5 <-> 10 <-> 20
print("\nSetelah insert_end:")
dll.display_maju()

# 4. Mencari data dengan target spesifik
print("\nPencarian dengan search:")
dll.search(20)

```

```

# 5. Pelepasan memori dari awal antrean
dll.delete_beg()    # List: 10 <-> 20
print("\nSetelah delete_beg:")
dll.display_maju()

# 6. Pelepasan memori dari akhir antrean
dll.delete_end()    # List: 10
print("\nSetelah delete_end:")
dll.display_maju()

```

Output:

```

... Setelah insert_beg:
Maju   : [5] <-> [10] <-> None

Setelah insert_end:
Maju   : [5] <-> [10] <-> [20] <-> None

Pencarian dengan search:
Data 20 ditemukan pada urutan ke-3.
Simpul dengan nilai 5 di awal list telah dihapus.

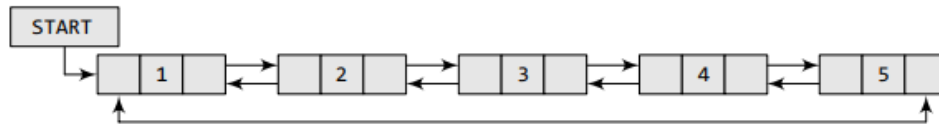
Setelah delete_beg:
Maju   : [10] <-> [20] <-> None
Simpul dengan nilai 20 di akhir list telah dihapus.

Setelah delete_end:
Maju   : [10] <-> None

```

F. CIRCULAR DOUBLY LINKED LISTS

Circular Doubly Linked List adalah struktur data kompleks yang menggabungkan kemampuan navigasi dua arah (maju dan mundur) dengan arsitektur cincin melingkar tanpa putus. Dalam struktur ini, setiap elemen (*node*) menyimpan referensi ke *node* sebelum dan sesudahnya, di mana *node* paling akhir akan menunjuk kembali ke *node* pertama (START), dan sebaliknya, *node* pertama menunjuk mundur ke *node* paling akhir. Karena desainnya yang saling mengunci tersebut, tidak ada nilai penunjuk kosong (NULL atau None dalam Python) di kedua ujung antreannya. Meskipun membutuhkan ruang memori lebih besar, bentuk Circular Doubly ini memberikan keuntungan besar dalam memanipulasi elemen secara fleksibel dan membuat operasi pencarian data menjadi dua kali lebih efisien.



Gambar 3. Circular Doubly Linked List

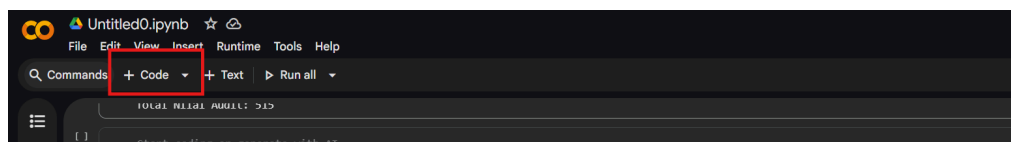
1. Inisialisasi

Struktur ini merupakan gabungan arsitektur dua arah (Doubly) dengan sifat melingkar (Circular). Oleh karena itu, kita tetap menggunakan arsitektur `class Node` yang memiliki tiga atribut (`prev`, `data`, `next`).

Inisialisasi awal pada `class CircularDoublyLinkedList` dimulai dengan menetapkan variabel `self.start = None`. Penetapan nilai kosong pada konstruktor ini sangat krusial murni sebagai kondisi dasar (*base case*) untuk menandakan bahwa antrian masih kosong.

Namun, begitu elemen pertama disisipkan ke dalam memori, sifat melingkar akan aktif: referensi `next` dan `prev` dari elemen tersebut tidak akan pernah lagi bernilai `None`, melainkan akan langsung menunjuk ke dirinya sendiri untuk mengunci cincin referensi.

silahkan buat 1 cell dan coba ketikan kode ini di google colab :



```
class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None

class CircularDoublyLinkedList:
```

```
def __init__(self):  
    self.start = None
```

2. Traversing (Penelusuran)

Pada *Circular Doubly Linked List*, penelusuran ini memiliki keunikan:

1. Dua Arah: Karena setiap *node* memiliki dua penunjuk (*next* dan *prev*), kita bisa menelusuri list ke arah depan (maju) maupun ke arah belakang (mundur).
2. Kondisi Berhenti (Tanpa None): Tidak seperti list biasa yang ujungnya bernilai None (atau NULL di C), list ini membentuk cincin yang saling mengunci. Penunjuk *next* pada *node* terakhir akan kembali menyimpan alamat *node* pertama (START). Oleh karena itu, agar tidak terjadi *infinite loop* (perulangan tanpa henti), penelusuran akan dihentikan ketika penunjuk telah berputar dan kembali ke *node* pertama (START)

a. Penelusuran Maju

Kunci utama dari penelusuran ini terletak pada kondisi berhentinya. Karena struktur ini berbentuk cincin melingkar yang ujungnya saling mengunci dan tidak memiliki nilai akhir yang kosong (NULL di C atau None di Python), maka penelusuran maju akan dihentikan ketika penunjuk telah bergerak satu putaran penuh dan kembali menunjuk ke *node* pertama (START)

Tambahkan code di bawah ini dalam cell yang sama dengan code kelas Node :

```
def display_maju(self):  
    # Cek apakah list kosong  
    if self.start is None:  
        print("List Kosong.")  
        return  
    ptr = self.start  
    print("Penelusuran Maju : ", end="")  
  
    # Mulai perulangan  
    while True:
```

```

        print(f"[{ptr.data}]", end=" <-> ")
        ptr = ptr.next # Geser ke node sesudahnya

        # Kondisi berhenti: jika ptr kembali menunjuk ke START
        if ptr == self.start:
            break

    print("(Kembali ke START)")

```

b. Penelusuran Mundur

Penelusuran mundur (*backward traversing*) pada Circular Doubly Linked List adalah proses menelusuri atau membaca isi *node* dari posisi paling belakang menuju ke depan secara berurutan menggunakan penunjuk referensi arah mundur (*prev*).

Tambahkan code di bawah ini dalam cell yang sama, setelah fungsi *display_maju* :

```

def display_mundur(self):
    if self.start is None:
        print("List Kosong.")
        return

    # Langsung lompat ke node paling akhir menggunakan
    self.start.prev
    # (Karena prev dari START selalu menunjuk ke node terakhir)
    ptr = self.start.prev
    print("Penelusuran Mundur : ", end="")

    while True:
        print(f"[{ptr.data}]", end=" <-> ")
        ptr = ptr.prev # Geser ke node sebelumnya

        # Kondisi berhenti: jika ptr kembali menunjuk ke node
        terakhir
        if ptr == self.start.prev:

```



```
break

print("(Kembali ke Akhir)")
```

3. Inserting (Penyisipan)

Saat menyisipkan elemen baru pada arsitektur yang circular dan doubly, perlakuan khusus harus diberikan untuk memastikan cincin tidak terputus.

a. Penyisipan di awal list

Saat *node* baru disisipkan di posisi START, kita tidak hanya menautkan *node* baru ke *node* START lama, tetapi kita juga harus melakukan penelusuran (traversing) untuk mencari posisi *node* paling akhir. Setelah *node* terakhir ditemukan, kita menautkan penunjuk next-nya ke *node* baru, dan referensi prev pada *node* baru diarahkan ke *node* terakhir tersebut. Selanjutnya, penanda START dipindah ke *node* baru

Tambahkan code di bawah ini dalam cell yang sama, sesudah code fungsi *display_mundur* :

```
def insert_beg(self, val):
    new_node = Node(val)
    if self.start is None:          # Jika list masih kosong
        new_node.next = new_node
        new_node.prev = new_node
        self.start = new_node
    else:
        ptr = self.start
        while ptr.next != self.start: # Cari node paling akhir
            ptr = ptr.next

        new_node.prev = ptr
        ptr.next = new_node
        new_node.next = self.start
        self.start.prev = new_node
        self.start = new_node
```

b. Penyisipan di akhir list

Iterasi dilakukan menggunakan bantuan variabel ptr dari START hingga menemui *node* terakhir. *Node* baru diletakkan setelah ptr. Penunjuk next dari *node* terakhir lama diarahkan ke *node* baru, lalu penunjuk next dari *node* baru dikembalikan ke START agar lingkarannya tertutup rapat

Tambahkan code di bawah ini dalam cell yang sama, sesudah code fungsi *insert_beg* :

```
def insert_end(self, val):
    new_node = Node(val)
    if self.start is None:
        self.insert_beg(val)
        return

    ptr = self.start
    while ptr.next != self.start:    # Cari node paling akhir
        ptr = ptr.next
    ptr.next = new_node
    new_node.prev = ptr
    new_node.next = self.start
    self.start.prev = new_node
```

4. Searching (Pencarian)

Proses pencarian pada Circular Doubly Linked List bekerja dengan membandingkan nilai yang dicari dengan setiap data di dalam *node* secara berurutan. Perbedaan utamanya dengan list biasa terletak pada kondisi berhenti. Karena list melingkar tidak memiliki penanda batas akhir berupa None (pengganti NULL di C), pencarian akan dihentikan saat penunjuk referensi sudah berputar dan kembali menunjuk ke *node* pertama (START)

Tambahkan code di bawah ini dalam cell yang sama, sesudah code fungsi *insert_end* :

```

#Fungsi pencarian maju (Forward Search)
def search(self, val):
    # Cek jika list kosong
    if self.start is None:
        print("List kosong. Pencarian dibatalkan.")
        return None

    ptr = self.start
    posisi = 1

    # Menggunakan while True untuk menyimulasikan iterasi melingkar
    while True:
        # Jika data pada node saat ini cocok dengan nilai yang
        dicari
        if ptr.data == val:
            print(f>Data {val} ditemukan pada urutan ke-{posisi}.")
            return ptr

        # Geser penunjuk ke node berikutnya
        ptr = ptr.next
        posisi += 1

        # Kondisi berhenti:
        # Jika penunjuk (ptr) sudah berputar dan kembali ke posisi
        START
        if ptr == self.start:
            break

    # Jika loop selesai dan data tidak ditemukan
    print(f>Data {val} tidak ditemukan di dalam list.")
    return None

```

5. Deleting

Penghapusan referensi harus dilakukan sedemikian rupa sehingga objek terlepas sempurna agar memori dibersihkan oleh sistem (*Garbage Collector* Python) tanpa memutus siklus antrean sisanya.

a. Penghapusan di awal list

Jika *node* START dihapus, kita kembali mencari lokasi *node* paling akhir menggunakan iterasi. Setelah ketemu, penunjuk next dari *node* terakhir langsung dihubungkan ke *node* kedua dari *list*. Posisi START kemudian digeser ke *node* kedua

Tambahkan code di bawah ini dalam cell yang sama, sesudah code fungsi *search* :

```
def delete_beg(self):
    if self.start is None:
        return

    ptr = self.start
    if ptr.next == self.start: # Kasus jika list hanya bersisa 1
node
        self.start = None
    else:
        while ptr.next != self.start: # Cari node paling akhir
            ptr = ptr.next

        ptr.next = self.start.next
        self.start.next.prev = ptr
        self.start = self.start.next
```

b. Penghapusan di akhir list

Iterasi mencari *node* terakhir (*ptr*). Untuk menghapus *ptr*, penunjuk next pada *node* kedua dari belakang (*ptr.prev*) ditautkan secara langsung memotong ke START. Demikian juga *START.prev* dikembalikan tautannya ke arah *node* kedua dari belakang

Tambahkan code di bawah ini dalam cell yang sama, sesudah code fungsi *delete_beg* :

```
def delete_end(self):
    if self.start is None:
        return
```

```

ptr = self.start
if ptr.next == self.start:
    self.start = None
else:
    while ptr.next != self.start: # Cari node paling akhir
        ptr = ptr.next

    ptr.prev.next = self.start
    # START menunjuk mundur menargetkan node kedua dari
    belakang
    self.start.prev = ptr.prev
    # Node terakhir (ptr) kehilangan tautan dan dihapus GC

```

6. Eksekusi Program

Sebagai tahap terakhir, kita akan memvalidasi keseluruhan operasi pada `CircularDoublyLinkedList` menggunakan pendekatan eksekusi yang sama. Pemanggilan fungsi penelusuran (maju dan mundur) secara berkala akan membuktikan secara visual apakah cincin referensi ganda telah terhubung dengan sempurna.

Silakan tambahkan blok pengujian ini di sel terbawah Google Colab Anda, tepat di luar area indentasi `class`:

```

# 1. Inisialisasi object
cdll = CircularDoublyLinkedList()

# 2. Masukkan data di awal
cdll.insert_beg(10) # List: 10
cdll.insert_beg(5)  # List: 5 <-> 10
print("Setelah insert_beg:")
cdll.display_maju()

# 3. Masukkan data di akhir
cdll.insert_end(20) # List: 5 <-> 10 <-> 20
print("\nSetelah insert_end:")
cdll.display_maju()
#4 Mencari data dengan value 20

```

```

print("\npencarian dengan search:")
cdll.search(20)

# 5. Hapus dari awal
cdll.delete_beg() # List: 10 <-> 20
print("\nSetelah delete_beg:")
cdll.display_maju()

# 6. Hapus dari akhir
cdll.delete_end() # List: 10
print("\nSetelah delete_end:")
cdll.display_maju()

```

Output Program :

```

... Setelah insert_beg:
Penelusuran Maju : [5] <-> [10] <-> (Kembali ke START)

Setelah insert_end:
Penelusuran Maju : [5] <-> [10] <-> [20] <-> (Kembali ke START)

pencarian dengan search:
Data 20 ditemukan pada urutan ke-3.

Setelah delete_beg:
Penelusuran Maju : [10] <-> [20] <-> (Kembali ke START)

Setelah delete_end:
Penelusuran Maju : [10] <-> (Kembali ke START)

```

G. STUDI KASUS

Studi Kasus: Daftar Nilai Siswa

Keterangan Soal

Seorang guru ingin menyimpan daftar nilai ujian siswanya, yang terdiri dari Nomor Induk Siswa (*nis*) dan Nilai Ujiannya (*nilai*). Karena jumlah murid yang mengikuti ujian susulan bisa bertambah sewaktu-waktu, guru tersebut tidak bisa menggunakan *Array* yang ukurannya tetap (*fixed size*). Anda diminta membuatkan struktur *Linked List* yang dinamis agar data murid baru bisa ditambahkan ke urutan paling belakang kapan saja tanpa batasan ukuran. Simulasikan dengan menyimpan 3 data siswa, misalnya: S01 (Nilai 78), S02 (Nilai 84), dan S03 (Nilai 45).

Kebutuhan

1. Sebuah cetak biru elemen (node) untuk menyimpan dua informasi sekaligus: NIS (Nomor Induk Siswa) dan nilai, serta satu penunjuk referensi next.
2. Struktur utama Singly Linked List untuk mengelola data.
3. Fungsi untuk menyisipkan data murid baru di akhir antrean (insert at the end).
4. Fungsi untuk menelusuri dan mencetak (traversing & displaying) seluruh daftar nilai dari murid pertama hingga terakhir

Langkah-langkah yang Harus Dilakukan

1. Buat class Node yang memiliki atribut *nis*, *nilai*, dan rujukan *next* yang diinisialisasi dengan None.
2. Buat class *StudentLinkedList* dengan variabel *self.start = None* sebagai penanda awal antrean yang masih kosong.
3. Buat fungsi *add_student(nis, nilai)*. Di dalamnya, buat objek node baru. Jika list kosong, jadikan ia START. Jika tidak, lakukan iterasi penelusuran sampai ke ujung (node terakhir) lalu tautkan node baru tersebut di sana.
4. Buat fungsi *display_students()* menggunakan looping while dari node START untuk mencetak *nis* dan *nilai* satu per satu hingga mencapai batas ujung yang bernilai None.
5. Di blok eksekusi, buat objek list, tambahkan 3 data murid, lalu panggil fungsi cetak.

Jawaban :

```
1 # 1. Class Blueprint untuk elemen siswa
2 class Node:
3     def __init__(self, nis, nilai):
4         self.nis = nis # Atribut penyimpan Nomor Induk
5         self.nilai = nilai # Atribut penyimpan Nilai
6         self.next = None # Penanda akhir referensi
7
8 # 2. Class Utama untuk Linked List
9 class StudentLinkedList:
10     def __init__(self):
11         self.start = None # Inisialisasi antrean kosong
12
13 # 3. Fungsi menambahkan data murid di akhir list
14 def add_student(self, nis, nilai):
15     new_node = Node(nis, nilai)
16
17     # Jika belum ada siswa sama sekali
18     if self.start is None:
19         self.start = new_node
20     else:
21         # Penelusuran (Traversing) mencari node paling terakhir
22         ptr = self.start
23         while ptr.next is not None:
24             ptr = ptr.next
25         # Tautkan node siswa baru di akhir antrean
26         ptr.next = new_node
27
28 # 4. Fungsi menampilkan seluruh data siswa
29 def display_students(self):
30     ptr = self.start
31     if ptr is None:
32         print("Daftar nilai masih kosong!")
33         return
34
35     print("=== DAFTAR NILAI SISWA ===")
36     # Looping untuk membaca data satu per satu sampai bertemu None
37     while ptr is not None:
38         print(f"Nomor Induk Siswa: {ptr.nis} \t| Nilai: {ptr.nilai}")
39         ptr = ptr.next # Geser penunjuk ke siswa berikutnya
40
41     print("-----\n")
42
43 # =====
44 # 5. BLOK EKSEKUSI PROGRAM
45 # =====
46 # Membuat objek list nilai siswa
47 student_list = StudentLinkedList()
48
49 # Memasukkan data nilai siswa (S01, S02, S03)
50 student_list.add_student("S01", 78)
51 student_list.add_student("S02", 84)
52 student_list.add_student("S03", 45)
53
54 # Menampilkan hasil
55 student_list.display_students()
```


Output :

```
*** === DAFTAR NILAI SISWA ===  
Nomor Induk Siswa: S01 | Nilai: 78  
Nomor Induk Siswa: S02 | Nilai: 84  
Nomor Induk Siswa: S03 | Nilai: 45  
-----
```

Pembahasan :

Studi kasus ini mencontohkan penggunaan paling dasar dari *Singly Linked List* yang realistis dan mudah dipahami. Pada bahasa C, kita memerlukan perintah memori kompleks seperti `malloc`, namun pada Python, kita bisa langsung membuat objek `Node("S01", 78)` yang menyimpan dua tipe data secara rapi. Logikanya sangat sederhana: jika ada murid susulan yang baru masuk, program cukup mencari posisi murid paling ujung (`ptr.next is not None`), lalu menyambungkan murid baru tersebut di urutan selanjutnya.

REFERENSI

Thareja, R. (2014). *Data Structures Using C* (2nd ed.). New Delhi, India: Oxford University Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Hoboken, NJ: John Wiley & Sons, Inc.

GeeksforGeeks. (n.d.). *Python Linked List*. Diakses pada 7 Maret 2026, dari <https://www.geeksforgeeks.org/python/python-linked-list/>

GeeksforGeeks. (n.d.). *Singly Linked List Tutorial*. Diakses pada 7 Maret 2026, dari <https://www.geeksforgeeks.org/dsa/singly-linked-list-tutorial/>

GeeksforGeeks. (n.d.). *Circular Linked List*. Diakses pada 7 Maret 2026, dari <https://www.geeksforgeeks.org/dsa/circular-linked-list/>

GeeksforGeeks. (n.d.). *Doubly Linked List*. Diakses pada 7 Maret 2026, dari <https://www.geeksforgeeks.org/dsa/doubly-linked-list/>